12.1 Tree 的等價定義

一個有限、非空的 simple undirected graph T（頂點數 $n \geq 1$）是 tree，若它連通且沒有 cycle。對 $n \geq 1$ 的有限無向簡單圖，下列敘述等價：

1. 連通且無 cycle。
2. 連通且恰有 $n - 1$ 條邊。
3. 無 cycle 且恰有 $n - 1$ 條邊。
4. 任兩個頂點之間恰有一條 simple path。

Forest（森林）只要求無環，可能有多個連通分量；不是任意無環圖都連通。單點圖是 n=1 的基例，恰有 0 邊，任意兩點的路徑唯一性在唯一點到自身的長度 0 path 上成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

樹的等價刻畫逐項證明

先證樹的最長路徑端點為葉。若端點還接到路徑外一點，可延長；若接到路徑內另一點，便形成 cycle。對 $n \geq 2$，刪一個葉及其邊後仍連通無環，以 n=1、e=0 為基例，歸納得 $e = n - 1$。每個 forest 分量分別套用，得到 $e = n - c$。

若 G 連通且有 cycle，刪 cycle 上一邊仍連通；重複到 spanning tree，故任何連通圖至少有 n−1 邊。若一開始恰 n−1 邊，便不可能曾刪掉一邊，故 G 無環。若 G 無環且恰 n−1 邊，由 forest 公式得 c=1，故連通。

若 G 為樹，兩點至少有一路徑。假設有兩條不同 simple paths，取第一次分岔到下一個共同點的兩段，便形成 cycle，矛盾。反之任兩點路徑唯一保證連通；若有 cycle，其上兩點可沿兩個方向連接，違反唯一性。這完成四個刻畫的等價性，沒有先假設待證的邊數公式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

葉數公式與有限非空的邊界

對 n≥2 的樹，令 n_i 為 degree=i 的點數。由 $\sum_i n_i = n$ 及 $\sum_i i n_i = 2(n-1)$，相減得到

$$n_1 = 2 + \sum_{i \geq 3} (i-2) n_i.$$

因此樹至少有兩個葉；degree 2 的點只細分路徑，不改葉數。例如 degree 3 的點有 2 個、degree 4 的點有 1 個，其餘非葉 degree 2，葉數必為 $2+2+2=6$。

單一頂點樹沒有 degree 1 的點；若採 rooted-tree 的「無 children」定義，它又是葉。因此上述公式明限 n≥2，並使用無向 degree 1 定義。空圖的 e=0、n=0 不能寫成 e=n−1；本章樹始終非空。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

12.2 Edge 的 cycle 等價

在 connected graph G 中，edge e 是 bridge，若且唯若 e 不在任何 cycle 中。因此 tree 的每條 edge 都是 bridge；每新增一條 edge 到 tree 必形成唯一 cycle（fundamental cycle）。

反例：三角形加一條 dangling edge，dangling edge 是 bridge，triangle edges 不是。Graph 可以有 bridges 也有 cycles；「tree」額外要求所有 edges bridge 且 connected。

移除 bridge 會增加 components；移除 non-bridge 不一定使 graph disconnected。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

12.3 有根樹：從根往下看層次

在一棵樹中指定一個頂點作為**根（root）**，就得到有根樹（rooted tree）。每個非根節點通往根的唯一路徑上，緊鄰它的上一個節點是父節點（parent）；反過來，它就是該父節點的子節點（child）。沒有子節點的節點稱為葉。

深度數邊，高度取最大深度。本章根的深度為 0；節點深度是根到它的路徑邊數。深度同為 d 的節點在第 d 層，整棵樹的高度是所有節點深度的最大值。因此單一節點樹的高度為 0，根連到兩個葉的樹高度為 1。

m 元樹（m-ary tree）每個節點至多有 m 個子節點；full m 元樹的每個內部節點恰有 m 個子節點。二元樹還區分左、右子節點：只有左子節點與只有右子節點，是不同的結構。以下公式均以根深度 0、用邊數計高度；若改用經過的層數，數值會多 1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

有序樹：用地址標出子節點的位置

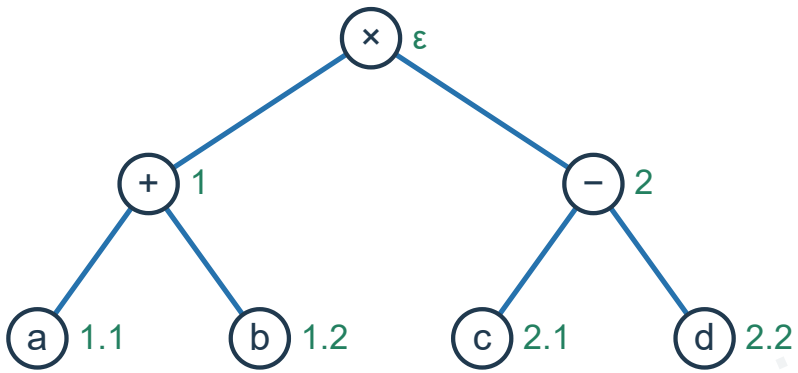
有根樹決定父子關係；有序有根樹（ordered rooted tree）另外固定每個節點的子節點次序。根的地址為空字串 ε；第 i 個子節點的地址，是在父節點地址後接上 i。例如地址 2.1 表示「根的第 2 個子節點，再往下取第 1 個子節點」；其父節點地址是 2，祖先地址是 ε、2。地址有幾段，深度就是幾；這些數字表示位置，不表示鍵值大小。

固定本例運算式 $(a + b) \times (c - d)$。根是乘法，左右 child 為 +、−；葉依序 a、b、c、d。Preorder 先根後 children，postorder 先 children 後根；binary inorder 則左子樹、根、右子樹。一般多元 ordered tree 沒有唯一的 inorder 慣例。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

有序樹：用地址標出子節點的位置 (續)

同一棵 ordered binary tree：地址決定子樹與運算次序



根地址 ε，depth 0；葉地址長度 2，depth 2。

Preorder：× + a b − c d

Inorder（加括號）：(a+b)×(c−d)

Postorder：a b + c d − ×

2、3、9、4 代入，結果為 25。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一棵樹的地址、走訪與運算式

節點	×	+	a	b	−	c	d
地址	ε	1	1.1	1.2	2	2.1	2.2
Depth	0	1	2	2	1	2	2

Preorder 為 `× + a b − c d`，是 prefix；postorder 為 `a b + c d − ×`，是 postfix。Inorder 加上子樹括號為 `(a+b)×(c−d)`；若把括號全刪掉，`a+b×c−d` 通常是另一個運算式，走訪序列不會自動保留優先序。

令 a=2、b=3、c=9、d=4。Postorder 到 + 時已知 2、3，得到 5；到 − 時已知 9、4，得到 5；最後乘法為 25。每節點做一次固定成本動作，每邊至多遞迴往返一次，所以三種走訪均 $\Theta(n)$，遞迴堆疊 $O(h+1)$，最壞斜樹為 $O(n)$。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

12.4 二元樹的節點數與高度

在 full binary tree 中，每個內部節點恰有兩個子節點。令內部節點數為 I、葉數為 L，則 $L = I + 1$。固定 root depth 0，若 height 至多 h，則

$$I \le 2^h - 1, \qquad L \le 2^h.$$

只有每層都填滿、全部葉都在深度 h 的 perfect binary tree 才同時取等。Full 本身不保證葉同深：根的一側是葉、另一側是一個有兩葉的內部點時，height=2、I=2、L=3；它是 full，卻不是 perfect，不能套 I=3、L=4。

這裡先計算一棵樹的節點數與高度；稍後的 Catalan 數則計算有多少種不同的有序樹形。兩者計數的對象不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Full、perfect、complete：三種條件各管什麼

三個名稱容易混淆，先看它們限制哪一部分：**full 限制子節點數**，每個內部節點恰有 m 個子節點；**perfect 再限制葉的深度**，所有葉都在同一層；**complete binary tree 限制填入次序**，除最後一層外都填滿，最後一層從左往右連續填入。不同教材的命名可能不同，判斷時以結構條件為準。

若 full m-ary tree 有 I 個內部點、L 個葉，$m \geq 2$，每條邊恰由一個 parent 指向 child，因此

$$mI = I + L - 1, \qquad L = (m-1)I + 1, \qquad n = mI + 1.$$

高度 h、每點至多 m children 時，第 d 層至多 m^d 點，總點數至多 $(m^{h+1}-1)/(m-1)$，葉數至多 m^h。葉數界可對高度歸納：根的至多 m 棵子樹各至多 m^{h-1} 葉。取等需每層滿到相同高度，不是只要 full 即可。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Binary tree 形狀與 Catalan 遞迴

令 C_n 為有 n 個內部點的 full ordered binary tree 形狀數。n=0 是單一葉，$C_0 = 1$。n≥1 時根為內部點；左子樹有 i 個內部點，右子樹必有 n−1−i 個，且分解唯一，所以

$$C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i}, \qquad C_0, C_1, C_2, C_3 = 1, 1, 2, 5.$$

這與第 10 章 Catalan 遞迴相同，給 $C_n = \frac{1}{n+1}\binom{2n}{n}$。以先序編碼每個內部點為（、每個葉為），再移去最後一個葉符號，得到 n 對合法括號。任何前綴的左括號數不少於右括號，因尚未填滿的 child slots 不能提前耗盡；反向按空 slot 逐一建節點可唯一還原。這是形狀計數的雙射，而非只因前幾個數值相同。

左右子樹交換一般算另一形狀；若取消 child 的順序，上述乘積分類便會重複計數。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

12.5 Comparison sorting decision tree

任何 comparison sort 的執行可由 binary decision tree 表示：internal node 問一個 comparison，leaf 是 output permutation。對 n 個相異輸入需分辨 n! 個 orders。

若 decision tree 有 L 個 leaves、height 為 H，binary tree 的 leaf bound 是 $L \le 2^H$。故

$$2^H \ge n!, \quad H \ge \log_2(n!) = \Omega(n \log n).$$

此 lower bound 適用 comparison model；counting/radix sort 使用更強 key assumptions，可超越比較界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

比較排序的最壞與平均下界

在相異鍵、只准兩路比較的模型，每個可達輸入次序通往一片不同葉。對數的下界可不用 Stirling 近似：$n! \geq (n/2)^{n/2}$，故最壞比較數至少 $\log_2(n!) = \Omega(n \log n)$。這是某個輸入的下界，不表示每個輸入都需此時間。

若 n! 種次序等機率，設其葉深為 $d_1, \ldots, d_N$，N=n!。Binary tree 的葉地址是 prefix-free，所以 $\sum_i 2^{-d_i} \leq 1$。由算術平均大於等於幾何平均，

$$2^{-\frac{1}{N}\sum_i d_i} \leq \frac{1}{N}\sum_i 2^{-d_i} \leq \frac{1}{N},$$

所以平均比較數也至少 $\log_2 N$。若比較結果允許 equality，但輸入鍵保證相異，equality 分支永不出現，仍只有兩個可達分支。若輸入次序不等機率，應用實際分布的熵界，不能直接沿用均勻平均的結論。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

12.6 Prefix code 定義

二進位編碼 C 先要求是從來源 symbols 到 bit strings 的**單射**：不同 symbols 必須有不同 codewords。在此前提下，若任一 codeword 都不是另一 codeword 的真 prefix，稱 prefix-free。等價地，對不同 symbols a、b，C(a) 不得是 C(b) 的 prefix，這裡包含相等情形。

單射條件不能省略：把 a、b 都編成 0，雖沒有兩個不同字串形成「真 prefix」，收到 0 仍無法知道是哪個符號。合法 prefix code 才能以 binary trie 的 leaves 解碼；每片葉標一個唯一 symbol，從 root 讀 bits 至該葉即輸出並回到 root。

固定 symbol frequencies f_i，平均長度

$$L = \frac{1}{\sum_i f_i} \sum_i f_i |w_i|.$$

Prefix-free 是保證唯一且可立即解碼的充分條件，不是所有 uniquely decodable code 的必要條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Kraft 不等式與立即解碼

有限 binary prefix code 的長度為 ℓ_i。設最大長度 H；長度 ℓ_i 的 codeword 占據深 H 完整二元樹的 $2^{H-\ell_i}$ 個後代葉。不同 codewords 的後代集合不交，故

$$\sum_i 2^{H-\ell_i} \le 2^H \quad \Longrightarrow \quad \sum_i 2^{-\ell_i} \le 1.$$

反向若給定非負整數長度且滿足此式，依長度非降排序，在樹中逐一取尚未被先前 codewords 遮蓋的節點。在深 ℓ 的層上，已占節點數為 $\sum_{j<i} 2^{\ell-\ell_j}$；包含本次要求的 Kraft 部分和不超過 1，保證至少剩一格。所取節點不在先前葉之下，往後也不再取它的後代，便建出 prefix code。

例長度 $1,2,3,3$ 的和為 $1/2+1/4+1/8+1/8=1$，可取 `0,10,110,111`。長度 $1,1,2$ 的和為 5/4，不可能。Kraft 不等式不保證任意給定的 codeword 字串已 prefix-free，例如 `0,00` 仍有 prefix 衝突。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

12.7 Huffman：每次合併兩個最小權重

給定至少兩個符號，各有正頻率 f_i。用二進位 prefix code 編碼，符號 i 的碼長等於葉深度 ℓ_i，要最小化的是加權外部路徑長度 $W = \sum_i f_i \ell_i$。頻率固定時，最小化 W 也就最小化平均碼長。

先讓每個符號各占一片葉。每輪從最小優先權佇列取出兩棵最輕的樹，以新父節點連接，將權重和放回佇列；直到只剩一棵樹，再把每對子邊標成 0、1。內部節點的權重是子樹頻率總和，不是新的來源符號。

樹高最小，不等於 W 最小。少見符號的路徑稍長，可能換來常見符號的路徑大幅縮短。要證明這種貪婪選擇正確，還須回答：為何兩個最小權重可以先當兄弟？把它們合起來後，為何可以只解較小問題？

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

交換引理：小權重往深處移，成本不增加

先排除只有一個子節點的內部節點：把它收縮掉，所有後代的深度減 1，正權重使 W 嚴格下降。因此最優樹可取為 full binary tree。取一片最深葉，深度為 H；它的兄弟也必為葉，否則兄弟的子節點會比 H 更深。

設 $f_x \le f_u$，x 原在深度 h，而 u 在 $H \ge h$。只交換這兩片葉上的符號，其餘項都不變，故

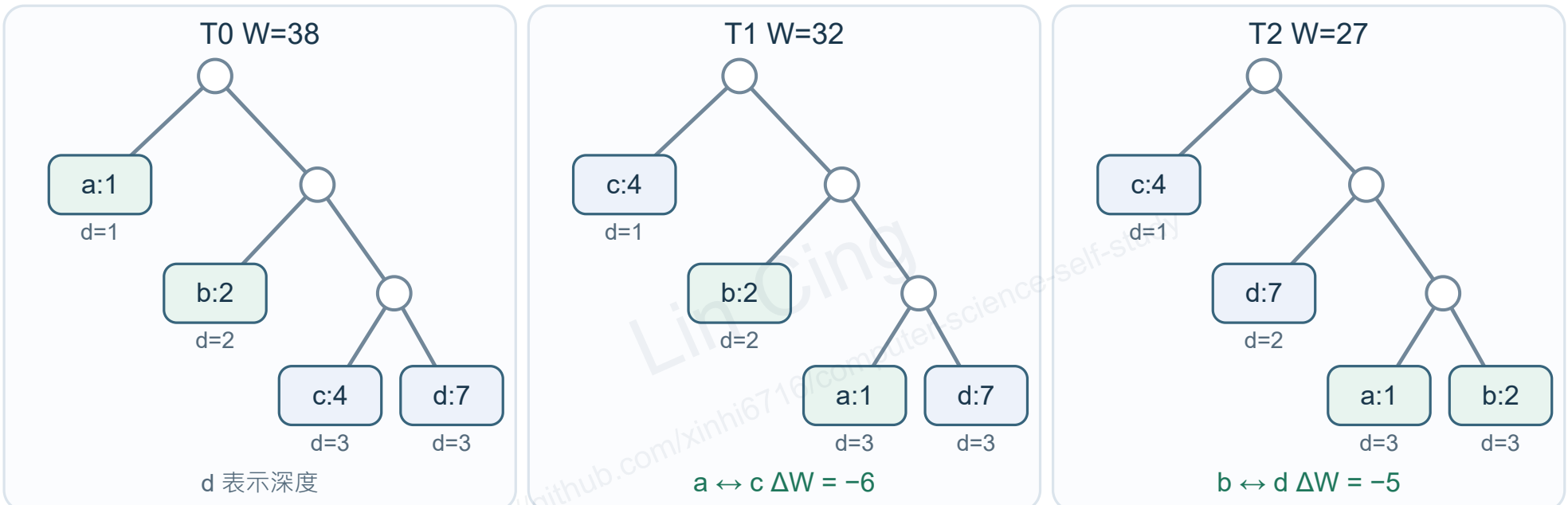
$$\Delta W = (f_x H + f_u h) - (f_x h + f_u H) = (f_u - f_x)(h - H) \le 0.$$

選兩個最小權重的符號 x、y，把它們放進這對最深兄弟位置。若其中一個已在位置上，就保留它，只換另一個；否則先放 x，再放 y。第二次不動 x，其他符號權重都不小於 f_y，所以兩次都能套用不等式。若起點是最優樹，成本不可能再降低，交換後便仍最優。結論是「存在符合這次合併的最優樹」，不是所有最優樹都長一樣。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

看得見的交換：38 → 32 → 27

只交換葉上的符號，樹形不變



這裡刻意從成本 38 的非最優樹開始。只換葉上的符號，連線與葉深度不變。最小的 a、b 最後成為深度 3 的兄弟；成本 27 仍高於 Huffman 的 24。交換引理保證不變差，不保證把任意樹交換兩次就已最優。

每次交換，究竟省在哪兩項？

頻率固定為 a:1、b:2、c:4、d:7。每格先列深度，再列「頻率 × 深度」的成本。

符號	原樹 T_0	a、c 交換後 T_1	b、d 交換後 T_2
a:1	深度 1；成本 1	深度 3；成本 3	深度 3；成本 3
b:2	深度 2；成本 4	深度 2；成本 4	深度 3；成本 6
c:4	深度 3；成本 12	深度 1；成本 4	深度 1；成本 4
d:7	深度 3；成本 21	深度 3；成本 21	深度 2；成本 14
W	38	32	27

第一次 a 多付 2，c 少付 8，合計 $\Delta W = (4-1)(1-3) = -6$。第二次 b 多付 2，d 少付 7，合計 $\Delta W = (7-2)(2-3) = -5$。交換權重相等或深度相等的兩葉時，差額為 0；符號仍須保持一對一編碼。

收縮兩片兄弟葉：只少付最後一層

將兄弟葉 x、y 及共同父節點視為一片新葉 z，令 $f_z = f_x + f_y$。若 z 在收縮後樹 U 的深度為 k，展開後 x、y 都在 k+1；其他葉的深度不變。

$$f_x(k+1) + f_y(k+1) = (f_x + f_y)k + f_x + f_y.$$

因此對任何以 x、y 為兄弟的樹 T，都有 $W(T) = W(U) + f_x + f_y$。反過來，任意較小問題的合法樹 U，都能在 z 的葉位置展開 x、y，得到原問題的合法 prefix tree；成本恰好多這個固定值。

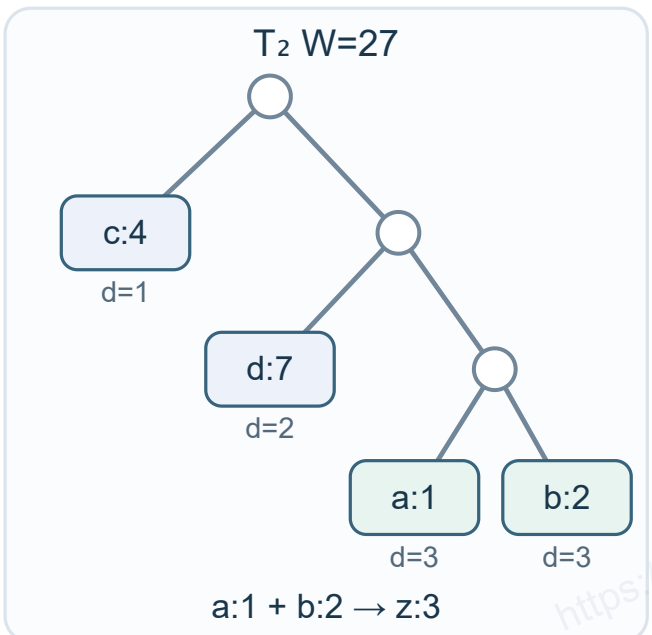
如果 T 已最優，U 也必最優。否則改用成本更低的 U*，再展開相同的 x、y，原問題成本也會降低，與 T 最優矛盾。這才是可以遞迴解較小問題的理由。

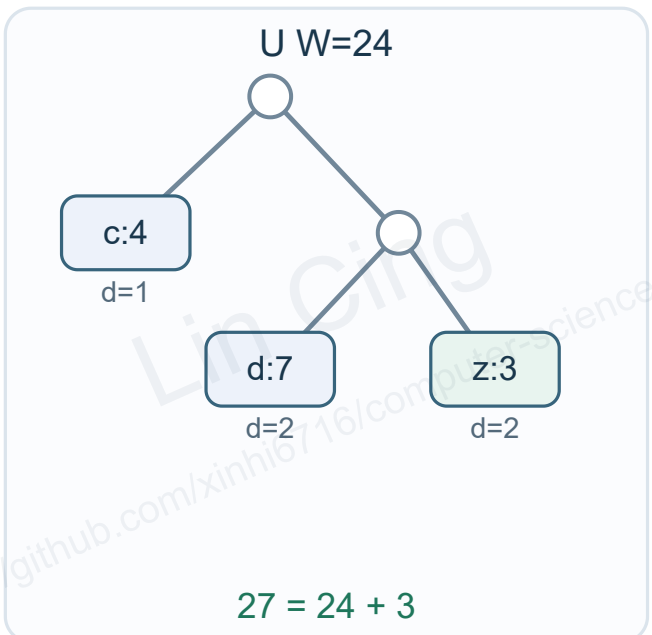
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

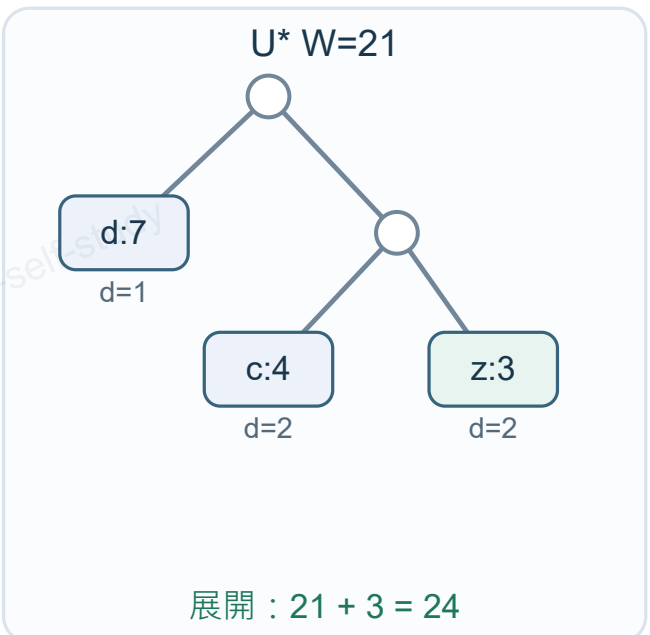
收縮數例：27 − 3 = 24，還能再改善

收縮保留了兄弟葉的共同父位置，較小問題仍需繼續最佳化。

先收縮，再把較小問題解到最優







Lin Cing
https://github.com/xinhi6716/computer-science-self-study

收縮後仍需求最優樹

T_2 的 a、b 在深度 3，收縮後 z:3 在深度 2。成本從 $1 \cdot 3 + 2 \cdot 3 = 9$ 變成 $3 \cdot 2 = 6$，故 U 的成本為 27−3=24。這是三符號問題的成本，不能直接當成原問題答案。

較小問題把 3、4 放在深度 2，7 放在深度 1，得到 $W(U^*) = 3 \cdot 2 + 4 \cdot 2 + 7 = 21$。展開 z，原問題成本為 21+3=24，a、b、c、d 的深度為 3、3、2、1。重新選擇樹形，才補上兩次交換尚未完成的改善。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從交換與收縮，完成最優性的歸納

固定正權重，對符號數 n 歸納。單一符號允許空字串時，成本為 0；兩個符號各用一位元，成本為兩權重之和。假設較少符號的 Huffman 樹均最優。

本輪選兩個最小權重 x、y，合成 z。交換引理保證存在原問題最優樹 T*，其中 x、y 是兄弟。收縮得到 U*；由前頁的反證，U* 必是較小問題的最優樹，因此

$$\mathrm{OPT}(C) = \mathrm{OPT}(C') + f_x + f_y, \qquad C' = (C \setminus \{x, y\}) \cup \{z\}.$$

Huffman 在 C' 上繼續合併，依歸納假設得到成本 $\mathrm{OPT}(C')$；展開 z 後就達到原問題的 OPT。每次合併使符號數少 1，故歸納會終止。相同最小權重可任選，0、1 邊可互換；定理不要求唯一碼字，也不要求葉的字母順序固定。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

12.8 Huffman 權重例

Frequencies $1, 2, 4, 7$。先合併 1+2=3，再取 3+4=7，最後 7+7=14。Weighted external path length 也可由所有 merge weights 相加：

$$3 + 7 + 14 = 24.$$

一種 code 可為 `1→110`,`2→111`,`4→10`,`7→0`；平均長度 weighted 後為 24/14。任何 prefix code 的 weighted cost 不低於此。

重建 codeword 時左右任選 0/1，只要解碼端使用同一棵 trie。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Huffman 的成本模型與邊界

每個符號頻率 $f_i > 0$，以 weighted external path length $W = \sum_i f_i \ell_i$ 為目標。最優樹不需 degree 1 的內部點，因收縮它可讓正權重後代深度全減 1，嚴格改善 W。因此至少兩符號時可在 full binary trees 中找最優解。

利用最小優先權佇列，每次取兩個最小、插入和，共 n−1 次合併，時間 $O(n \log n)$、樹與佇列 $O(n)$。若輸入已排序，可用原葉與新內部點兩個非降佇列，每次在兩個隊首選最小，合併總時間 $O(n)$，但不能漏算初始排序成本。

單一符號時數學模型可給空字串、成本 0，解碼需另知符號次數；若格式要求非空 codeword，約定 `0`，此時成本為 f。零頻率符號不影響目標，宜先移除或明定其編碼需求，不能套嚴格改善的正權重論證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

合併權重為何等於外部路徑成本

對葉 i，其頻率 f_i 在每個祖先的 subtree weight 中出現一次，恰有 ℓ_i 個祖先。因此按葉加總與按內部點加總可互換：

$$\sum_{i} f_i \ell_i = \sum_{\text{internal } v} \text{weight}(v).$$

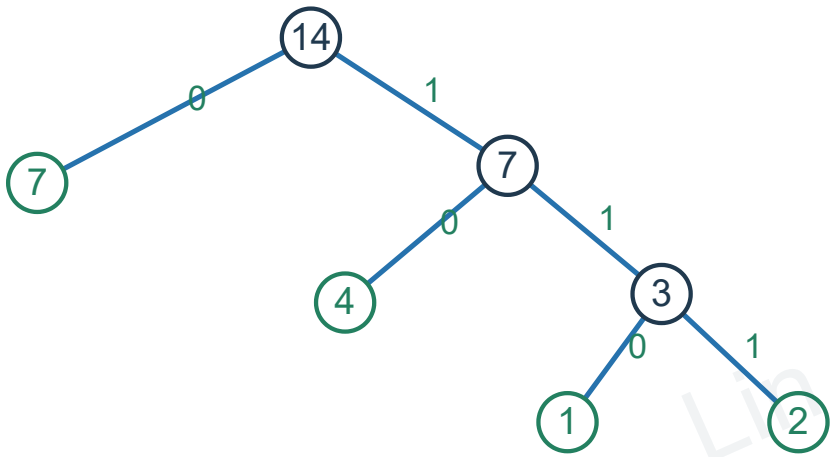
本章頻率 1、2、4、7 的合併表為

輪	取出的兩個最小值	插入值	佇列剩餘值
1	1、2	3	3、4、7
2	3、4	7	7、7
3	7、7	14	14

code `110,111,10,0` 的深度 3、3、2、1，成本 $1\cdot 3 + 2\cdot 3 + 4\cdot 2 + 7\cdot 1 = 24$，也等於 $3 + 7 + 14$。平均每個符號為 $24/14 \approx 1.7143$ bits，低於固定兩位元的 2，但尚未計入儲存 codebook 的成本。

Prefix code 的實際樹形

Huffman 的同一組權重、codewords 與 weighted cost



權重	code	長度	成本
7	0	1	7
4	10	2	8
1	110	3	3
2	111	3	6

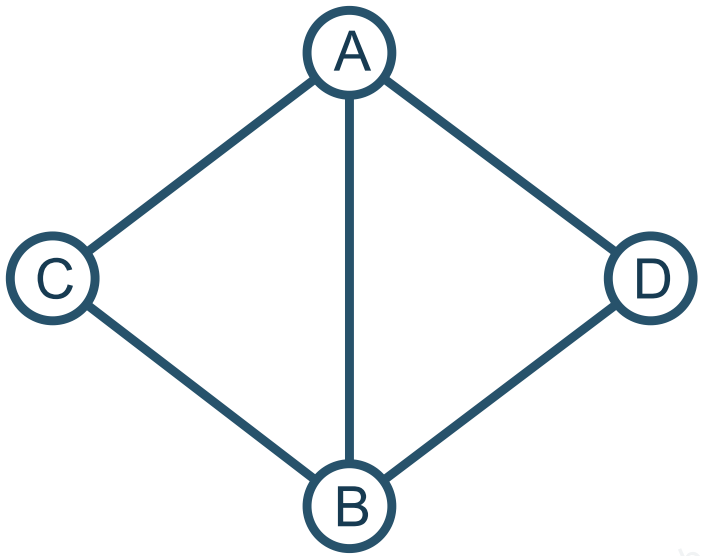
總成本 24；平均長度 24/14 bits。

每個內部節點標子樹總頻率，每條邊標一個 bit，只有葉代表來源符號。從 root 到葉的 bit 串就是 codeword。內部節點 7 與葉符號頻率 7 可以數值相同，但不能把內部節點也當作可解碼符號；否則其 bit 串會成為後代 codeword 的 prefix。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

生成樹數量與 Laplacian

以下以 K_4 刪去邊 CD 後的圖為例：計算包含全部四個頂點且無環的生成樹有幾棵。



五條邊

AB, AC, AD, BC, BD

A、B、C、D 的度數: (3, 3, 2, 2)

CD 已刪除

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

矩陣樹定理與關聯矩陣

對具有 $n \geq 1$ 個頂點的有限簡單無向圖，任意指定每條邊的方向，令 incidence matrix B 的每欄在尾端為 -1、首端為 $+1$，其餘為 0。Laplacian 是

$$L = BB^{\mathsf{T}} = D - A.$$

矩陣樹定理：刪去 L 的任一列及同編號欄後得到的主子矩陣 $L^{(r)}$，其行列式等於生成樹數 $\tau(G)$。不連通圖的值為 0；單點圖採空矩陣行列式為 1。

方向只是建立 B 的符號慣例；反轉一條邊會把其欄乘 -1，不改變 BB^{T}。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為何 cofactor 會計算生成樹？

刪去 B 的第 r 列，得到 $(n-1)\times m$ 矩陣 B_r，故 $L^{(r)} = B_r B_r^{\mathsf{T}}$。Cauchy–Binet 公式按 $(n-1)$ 條邊的集合 S 求和：

$$\det L^{(r)} = \sum_{|S|=n-1} \det(B_r[:, S])^2.$$

若 S 含環，沿環依方向賦予 ± 1，相應欄的和為零，故子式為 0。若 S 無環但不連通，因森林有 $n-c$ 條邊，無法同時滿足 $|S| = n-1$。因此非生成樹的 S 均不貢獻。

若 S 是樹，反覆選不是根 r 的葉，以其所在列展開行列式：該列只有一個非零項 ± 1。刪去該列和對應邊欄後仍為樹，最後得到空行列式 1。因此每棵生成樹貢獻 $(\pm 1)^2 = 1$，總和恰為生成樹數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

實算：K_4 刪去 CD

依頂點順序 A, B, C, D，五條邊為 AB, AC, AD, BC, BD。圖中的度數是 $3, 3, 2, 2$，因此

$$L = \begin{pmatrix} 3 & -1 & -1 & -1 \\ -1 & 3 & -1 & -1 \\ -1 & -1 & 2 & 0 \\ -1 & -1 & 0 & 2 \end{pmatrix}, \qquad L^{(D)} = \begin{pmatrix} 3 & -1 & -1 \\ -1 & 3 & -1 \\ -1 & -1 & 2 \end{pmatrix}.$$

沿第一列展開，三個帶符號的貢獻依序為

$$3(6-1) \;-\; (-1)(-2-1) \;+\; (-1)(1+3) = 15 - 3 - 4 = \boxed{8}.$$

因此原圖有 8 棵生成樹。這是對同一張五邊圖的計數；不能把完整 K_4 的生成樹數直接代入。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

12.9 Tree sorting 與 search tree 邊界

Binary search tree（BST）若維持 BST invariant，中序遍歷會把 keys 依排序次序列出：

$$\text{左側 keys} < key < \text{右側 keys}$$

（strict convention; duplicate policy另訂）。若插入有序資料且不 balance，height 可達 n−1，search worst-case Θ(n)。

Balanced tree 可把 height 控制到 Θ(log n)，但旋轉與平衡條件需正確維護，不能只憑「看起來平衡」。

Decision tree lower bound 說明 comparison sort 不能在 worst-case 低於 log(n!)；它不自動說每個 sorting tree 都能達成最優。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

DFS 與 BFS 建立的都是生成樹

在無向圖每個分量選一個根，首次發現新頂點時保留其發現邊，每個非根恰有一個 parent；發現時間嚴格沿 parent 增加，故沒有環，且連到根。因此得到 spanning forest，原圖連通時是一棵生成樹。

DFS 先遞迴完成一個鄰居子樹再換下一個；BFS 使用 FIFO queue，依非降距離處理。BFS 發現 w 時指定 $d[w] = d[v] + 1$：若有更短路徑，其倒數第二點會較早出隊並更早發現 w，矛盾。因此無權圖 BFS 深度就是最短距離；DFS 深度通常不是。

以邊集 $AB, BC, CA, CD, DE, EF, FD, DG$，每點鄰居按字母序：

方法	發現次序	樹邊
DFS	A、B、C、D、E、F、G	AB、BC、CD、DE、EF、DG
BFS	A、B、C、D、E、F、G	AB、AC、CD、DE、DF、DG

兩者發現次序恰相同也不代表樹相同；BFS 中 C 的深度為 1，DFS 中為 2。鄰接表時間 $O(n + m)$，鄰接矩陣則每點需掃 n 格，最壞 $\Theta(n^2)$。

無向 DFS 不存在跨兄弟子樹的邊

若原邊 xy 不是 tree edge，不妨 x 先被發現。若 y 在 x 的遞迴返回前仍未被發現，掃到 xy 就會把 y 放進 x 的子樹。若 y 更早已被發現，y 不是仍在堆疊中的祖先，就屬已完成的子樹；但該子樹先前掃到 yx 時理應發現 x，與 x 後來才被發現矛盾。因此每條非樹邊都連接祖先與後代。

這是 low-link 證明的核心。不同 DFS 子樹之間若能避開共同 parent 相連，必須靠回到祖先的 back edge，而不是任意的 cross edge。有向 DFS 可能有 cross edges，不能把本章公式直接搬去有向 strongly connected components。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Merge 的不變量與完整合併

已排序列表 L、R，保留各自游標 i、j，輸出 O 初始空。每次比較兩個尚未輸出首項，把較小者輸出並前進其游標；相等時先取左側即可維持 stability（等鍵的原順序）。其中一側耗盡，直接附上另一側剩餘內容。

不變量是 O 已排序，且恰為兩列表已移除項目的 multiset；所有尚未輸出項目都不小於 O 的最後一項。兩個首項的較小者一定是全部剩餘項目的最小值，故每步維持不變量。每步移除一項，剩餘總長度嚴格減少，最後得到完整排序合併。

例如合併 [2,3,6,8] 與 [1,4,5,7]，比較後依序輸出 1、2、3、4、5、6、7，再附上 8。前七項各用一次比較，總共 7 次；不是每次都掃整個剩餘列表。兩側長度 p、q 都正時，最壞比較數 p+q−1；最後一項無須再比。若其中一側初始為空，比較數為 0。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Merge sort 的分解、合併與比較次數

將列表拆成長度相差至多 1 的兩半，遞迴排序後再 merge。長度 0、1 直接返回。正確性對長度歸納：兩半較短而已各自排序，前述 merge 引理保證合併後完整且排序。

階段	實際資料狀態	本階段比較次數
輸入	[8,3,6,2,7,1,5,4]	0
合併單項	[3,8]、[2,6]、[1,7]、[4,5]	4
合併兩項	[2,3,6,8]、[1,4,5,7]	3+3=6
最後合併	[1,2,3,4,5,6,7,8]	7

總共 17 次比較。當 n=2^h，每層合併總比較數至多 $n - 2^d$，d=0 為根層，因此

$$M(n) \le \sum_{d=0}^{h-1} (n - 2^d) = n \log_2 n - n + 1.$$

Merge sort 的分解、合併與比較次數 （續）

本例 n=8 恰達 17。對一般 n，不可把 $\log_2 n$ 當整數高度；每層總處理量仍為 O(n)，最大深度 $\lceil \log_2 n \rceil$，得 $O(n \log n)$。配合比較排序下界，最壞成本為 $\Theta(n \log n)$。重用單一暫存陣列可用 O(n) 額外空間；若每層保留所有歷史副本則不是此空間模型。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

12.10 Articulation point 與 low-link

Vertex v 是 articulation point（cut vertex），若刪除 v 後 connected component 數增加。DFS discovery time $disc[v]$ 與 low value：

$$low[v] = \min\Big(disc[v], \min_{back\,edge\,v \to u} disc[u], \min_{tree\,child\,w} low[w]\Big).$$

Root 是 articulation，若且唯若有至少 2 個 DFS children；非 root v 是 articulation，若且唯若存在 tree child w 使 $low[w] \geq disc[v]$。

公式依 DFS back edge 處理；undirected edge 的 parent edge 不能誤算成 back edge。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

分離點與橋的兩種不等式

非根 v 有子樹 w，若 $low[w] \geq disc[v]$，該子樹沒有回到 v 的真祖先的邊。由無跨子樹邊性質，移除 v 後它與 parent 一側分離，所以 v 是 articulation。反之若每個 child w 都有 $low[w] < disc[v]$，每個 child 子樹均可經祖先回到 parent 一側；刪 v 後仍可互通，因此 v 不是分離點。

根沒有 parent 一側。若有至少兩個 DFS children，子樹間沒有跨邊，刪根後分離；若只有一個 child，所有其餘點在該棵連通子樹內，刪根不增加分量。孤立根刪除後只減少分量，也不是分離點。

對樹邊 vw，橋的條件更嚴格：$low[w] > disc[v]$。若等號，存在回邊連到 v，可繞過 vw，但不能繞過 v，所以 vw 不一定是橋而 v 仍可能是分離點。二者的 $>$、$\geq$ 不可交換。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一般圖的 low 值與 block 邊界

使用前面同一八邊圖；edge ID 固定為 1:AB、2:BC、3:CA、4:CD、5:DE、6:EF、7:FD、8:DG。從 A 按字母序 DFS。

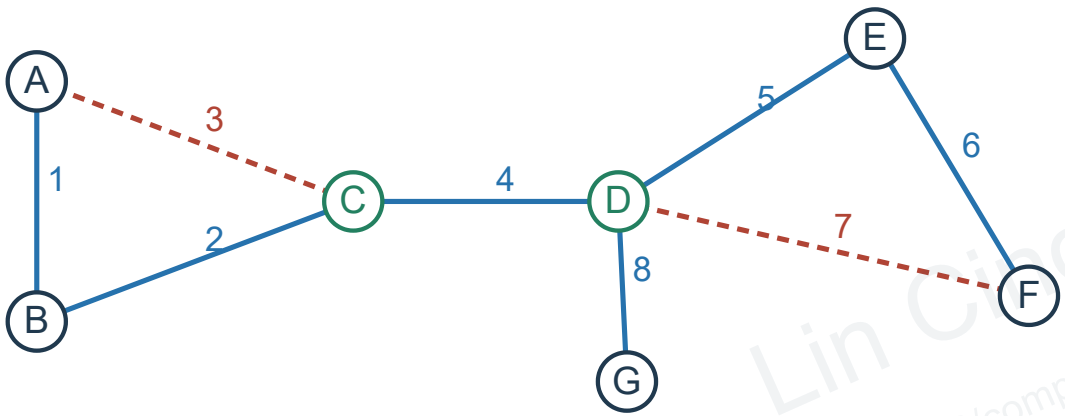
頂點	A	B	C	D	E	F	G
disc	1	2	3	4	5	6	7
low（返回後）	1	1	1	4	4	4	7
parent	無	A	B	C	D	E	D

CA 使 $low[C] = 1$，再傳回 B；FD 使 $low[F] = 4$，再傳回 E。G 沒有回邊，$low[G] = 7$。所以分離點為 C、D，橋為 CD、DG，根 A 雖滿足某個 child 的 $low \geq disc[A]$，仍只有一個 child，不能標為分離點。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一般圖的 low 值與 block 邊界 （續）

同一八邊圖的 DFS：兩個三角形、兩條橋與 low 值



頂點	A	B	C	D	E	F	G
disc	1	2	3	4	5	6	7
low	1	1	1	4	4	4	7

分離點：C、D；A 只有一個 child。

橋：4=CD、8=DG。

虛線：回邊 3=CA、7=FD。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Stack 實際內容與四次完整彈出

以下 stack 以 edge ID 表示，左底右頂。回邊只在後代指向祖先時推入一次，不能從兩端重複推入。

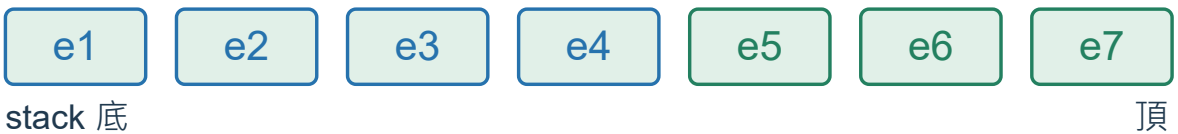
事件	stack（事件前或加入後）	本次 pop 次序	剩餘 stack
進入 E 的子樹並發現 FD	1,2,3,4,5,6,7	無	不變
E 返回 D：low[E]=4≥4	1,2,3,4,5,6,7	7,6,5	1,2,3,4
推入 DG，G 返回 D：7≥4	1,2,3,4,8	8	1,2,3,4
D 返回 C：4≥3	1,2,3,4	4	1,2,3
C 返回 B：1<2	1,2,3	無	1,2,3
B 返回 A：1≥1	1,2,3	3,2,1	空

得到 blocks：{DE,EF,FD}、{DG}、{CD}、{AB,BC,CA}。前三次與最後一次都包含觸發條件對應的 tree edge；根即使不標 articulation，仍必在 low 條件成立時輸出 block。

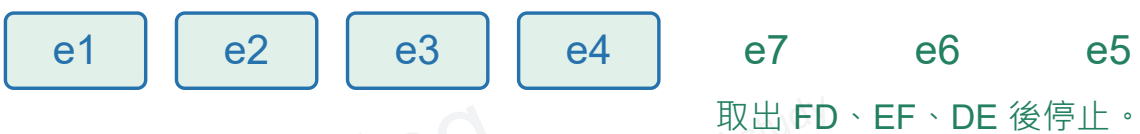
一次 pop 的前態、操作與後態

E 返回 D：彈出到 tree edge DE，保留較早分量的 stack

前態：low[E]=4，disc[D]=4



操作：連續 pop e7、e6、e5



後態：輸出完整 DEF block



保留 e1、e2、e3、e4

觸發的 tree edge 是 e_5=DE，所以「直到且包含 e_5」是停止條件。只 pop 最頂端 e_7=FD 會把三角形錯拆；一路把 e_4=CD 也取出，又會把連到較早部分的橋混入。這個操作不修改原圖，只把已完成的 block 從待分配 edge stack 輸出。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

12.11 Biconnected components

在有限無向圖中，block（vertex-biconnected component）定義為「連通且沒有 articulation point」的 maximal subgraph；maximal 以原圖的子圖包含關係判定。連通性是定義的一部分，不能納入不連通但沒有分離點的子圖。本章將橋的兩端與該邊 K_2 視為一個 block，孤立點 K_1 另作 singleton block。DFS 的 edge stack 會推入每個 tree edge 與指向祖先的 back edge。當 tree edge vw 的子樹完成，且

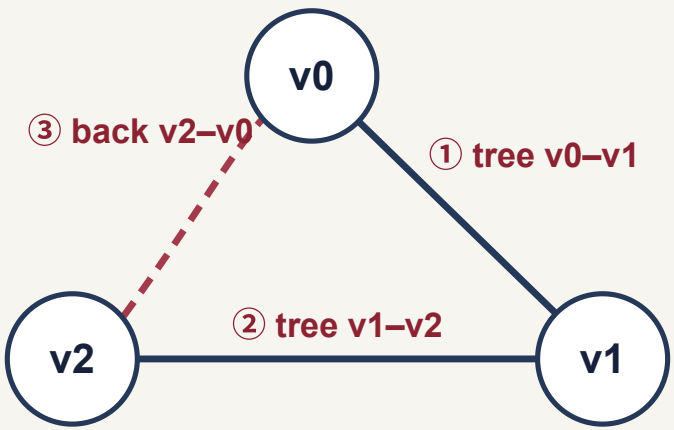
$$low[w] \geq disc[v],$$

就從 stack 頂端連續彈出 edge，直到彈出並包含這條 tree edge vw。彈出的整組 edge 才形成一個 block；只取其中一條不能代表整個 block。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

12.11 Biconnected components （續）

三角形 DFS：同步 disc 與 low



DFS 狀態（依 visit order）

- v0 disc=1 low=1
- v1 disc=2 low=1
- v2 disc=3 low=1

低值皆追溯至 root v0

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

12.11 block 的完整 stack 彈出

取三角形 $v_0v_1v_2$，從 v_0 開始，DFS 依序到 v_1, v_2，再由 v_2 走回祖先 v_0。實際狀態如下。

步驟	動作	disc	low	edge stack 內容（左為底）	彈出
1	訪問 v_0	$disc(v_0) = 1$	$low(v_0) = 1$	空	無
2	樹邊 v_0v_1	$disc(v_1) = 2$	$low(v_1) = 2$	(v_0, v_1)	無
3	樹邊 v_1v_2	$disc(v_2) = 3$	$low(v_2) = 3$	$(v_0, v_1), (v_1, v_2)$	無
4	回邊 v_2v_0	不變	$low(v_2) = \min(3, 1) = 1$	加入 (v_2, v_0)	無
5	回到 v_1	不變	$low(v_1) = \min(2, 1) = 1$	不變	$1 < disc(v_1) = 2$，不彈出

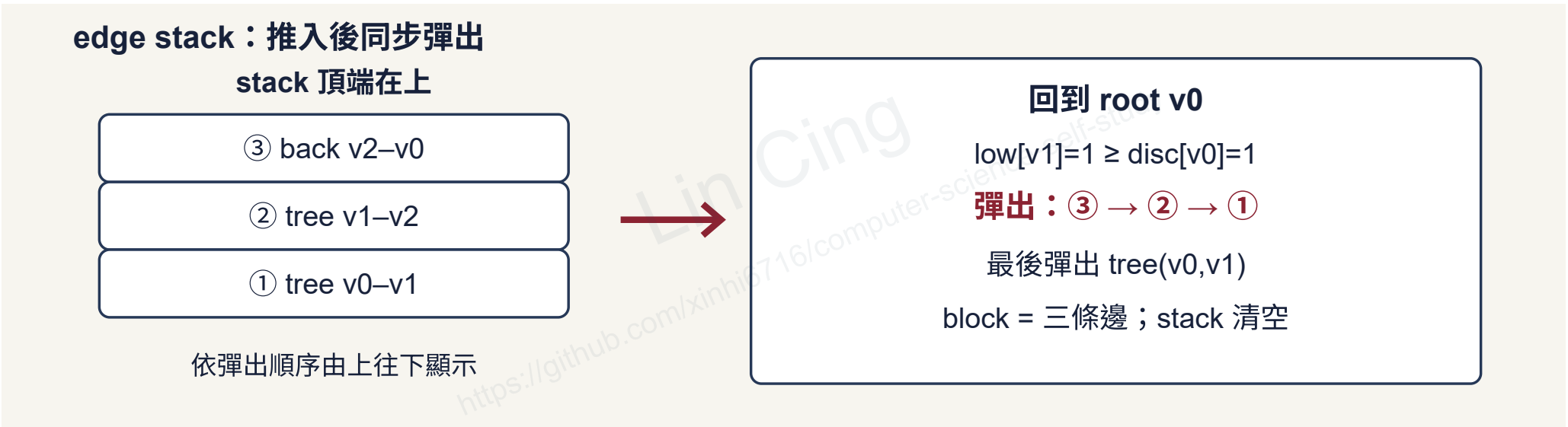
步驟	動作	disc	low	edge stack 內容（左為底）	彈出
6	回到 v_0	不變	$low(v_0) = 1$	不變	$1 \geq disc(v_0) = 1$，連續彈出三邊

12.11 stack pop 與 block 邊界

第 6 步的彈出順序是

$$(v_2, v_0),\ (v_1, v_2),\ (v_0, v_1),$$

最後彈出的元素正是 tree edge v_0v_1。這組三條 edge 構成完整 block；此時 stack 回到空。



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

12.11 stack pop 與 block 邊界 （續）

邊界正確性的關鍵是 stack suffix（從頂端到記得的 tree edge 的連續後綴）。每個尚未完成的 DFS 子樹只使用自己的後綴；若 $low[w] < disc[v]$，子樹仍可透過 back edge 連到 v 的祖先，故與外層留在同一 block；若 $low[w] \geq disc[v]$，子樹沒有連到 v 祖先的邊，移除 v 後便與外層分離。因此從頂端彈到 tree edge vw 恰好取出這個 block 的全部邊，先前已完成的 block 也不會被納入。此論證對一般深度的 DFS 成立，三角形只是逐步追蹤的實例。

Bridge 本身形成只有一條 edge 的 block；isolated vertex convention 需明示。

程式實作中 stack 記錄 edges，不是只記錄 vertices；否則跨 articulation 的 blocks 難以正確切分。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

完整的 DFS block 分解程序

本章圖為有限無向簡單圖。若要接受平行邊，必須以 edge ID 排除 parent edge，不能把所有通往 parent 的邊都跳過。以下版本直接使用 edge ID。

外層對每個未訪問點啟動 DFS。單一孤立點另輸出 singleton vertex block；自環不在本章輸入域。不能只在 cut[v] 為真時才 pop，因單一 DFS child 的根仍需輸出最後一個 block。

每次回傳時，以 low 值判斷割點、橋與 block 的邊界；以下程式逐條實作這些判準。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

DFS：掃描邊與遞迴

全域的 disc、low、cut 與 edge stack 跨呼叫保留；`finish_child` 處理一次子樹返回。

```
DFS(v, parent_edge):
    disc[v]=low[v]=++clock; children=0
    for (w,e) in adjacency[v]:
        if e == parent_edge: continue
        if disc[w] 未定:
            children++; stack.push(e); DFS(w,e)
            finish_child(v,w,e,parent_edge)
        else if disc[w] < disc[v]:
            stack.push(e); low[v]=min(low[v],disc[w])
    if parent_edge 不存在: cut[v]=(children>=2)
```

回邊只從後代端推入；edge ID 使另一條平行邊仍可回到父點。

子樹返回：橋與 block 邊界

`finish_child(v,w,e,parent_edge)` 接收剛完成的樹邊 e=vw；stack 在整棵 DFS 中共享。

```
finish_child(v,w,e,parent_edge):
    low[v]=min(low[v],low[w])
    if low[w] >= disc[v]:
        if parent_edge 存在: cut[v]=true
        B=[]
        repeat: z=stack.pop(); B.append(z)
        until z==e
        emit B
    if low[w] > disc[v]: emit bridge e
```

彈出包含 e；根是否為割點由 DFS 中的 child 數決定，不影響輸出 block。

為何輸出的是 maximal blocks

Stack 只放已發現、尚未分配給完成 block 的邊。每次遞迴從一條 tree edge 進入，該子樹新增的待處理邊都形成 stack 的連續後綴；先完成的內部 block 已被移除，不會再次納入。

當 $low[w] \geq disc[v]$，子樹 w 只能經 v 連到較早部分。任何跨過此界的更大連通子圖，一旦刪 v 就分離，因此不能把本次後綴與外部邊合成無分離點的 block。若 $low[w] < disc[v]$，存在回到真祖先的路徑，此時尚未到 maximal block 的上界，所以保留後綴。

彈出的集合內也不能再有分離點 z。若 z 在其內且不是上界 v，DFS 的分離點判準會找出其 child 子樹與其餘部分分開；那組邊早在該 child 返回時已按 $low \geq disc[z]$ 彈出，不會留到現在。若 z=v，本次集合只含一個 child 子樹連同 v；刪 v 後此子樹仍由 DFS tree 連通。故本次集合無分離點；結合外部不能合併，證得 maximality。橋的單邊集合依慣例也是 block。

每條邊只推入、彈出一次，鄰接項也只掃一次，所以時間 $O(n+m)$，額外 stack 與陣列 $O(n+m)$。這個計費包括輸出全部 block 邊，不是只計算 cut set。

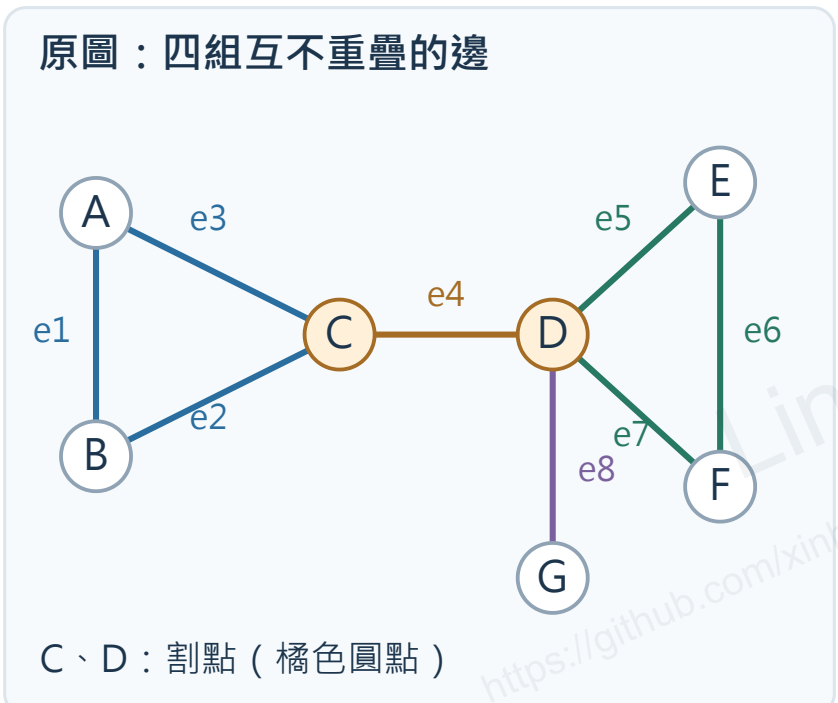
Block-cut tree 與多重歸屬

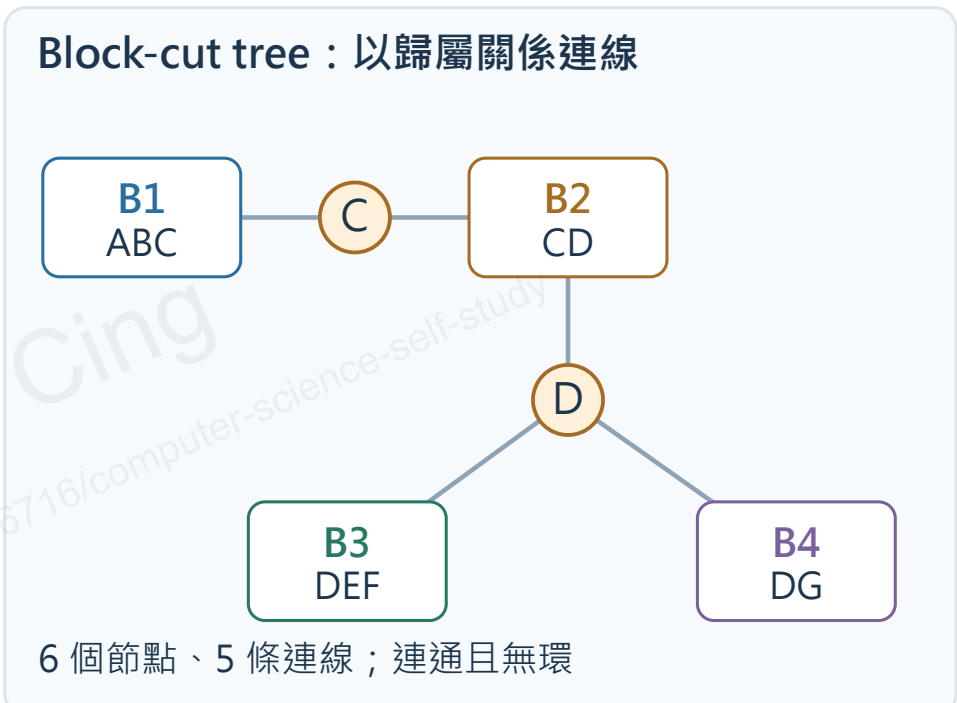
對每個 block 建一個 B 節點，對每個 articulation 建一個 cut 節點；cut v 屬於 block B 時連線。原圖連通時，此二分 incidence graph 連通。若它有環，沿環可繞過每一個宣稱的 cut 點，把相鄰 blocks 合成更大無分離點子圖，違反 maximality，所以它是一棵樹。一般圖得到 block-cut forest。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從原圖對照 block-cut tree

四個 blocks 為 B_1=ABC、B_2=CD、B_3=DEF、B_4=DG。右圖每個矩形代表一整個 block；橘色圓點代表割點。

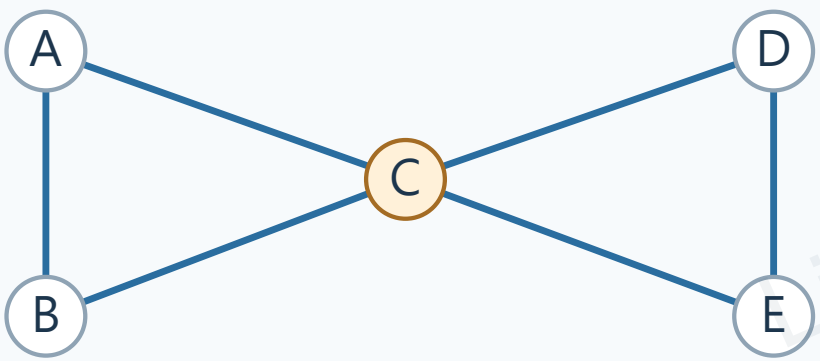




C 屬於 B_1、B_2；D 屬於 B_2、B_3、B_4，因此右樹共有 6 個節點、5 條歸屬連線。C、D 可以同屬多個 blocks，每條原圖邊卻只屬一個 block。右圖的連線表示「屬於」，不是原圖的一條邊。

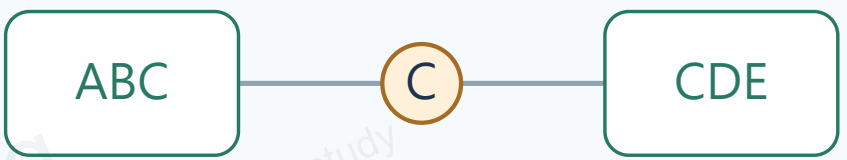
沒有橋，為什麼仍可能有割點？

每條邊都在某個環上



沒有橋；刪掉 C 後，AB 與 DE 分離

兩個 vertex blocks、一個割點



刪除所有橋：原圖沒有變，仍是一個分量。

依割點 C 分解：得到兩個 vertex blocks。

不重邊的連通性，與刪頂點的連通性不同。

刪任一邊，其兩端仍能沿同一三角形繞行；刪 C 卻會留下互不連通的 AB 與 DE。所以 C 是割點，但沒有任何邊是橋。刪橋得到一個 edge-biconnected component；依頂點分解則得到 ABC、CDE 兩個 blocks，共用 C。

12.12 反例與本章檢查

- 有限 simple graph 若 connected 且有 n−1 edges，即可推出 acyclic；不需額外假設。
- Prefix-free 不等於 codeword length 相等。
- Huffman 在 symbol frequencies 固定且 prefix code family 內最小化，跨類型 compression 要另比。
- Low-link 的根節點使用 child count，不是 parent 條件。
- Block-cut tree 記 blocks 與 articulation nodes，不可把原 graph edge 直接當 block。

參考資料與署名

Lin Cing，《離散數學》手寫筆記。 Ralph P. Grimaldi, Discrete and Combinatorial Mathematics, 5th ed., Ch. 12.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study